CMake

Contents

CMake has evolved significantly, and modern CMake (3.12+) emphasizes target-based configuration over global variables. This cheatsheet covers contemporary best practices for building C++ projects, managing dependencies, and creating reusable packages.

CMake CLI Quick Reference

Common command-line operations for configuring, building, and managing CMake projects.

Configure and Build

```
# Configure (generate build system)
cmake -B build                      # Configure into build/
cmake -B build -S src               # Specify source directory
cmake -B build -G Ninja             # Use Ninja generator
cmake -B build -G "Unix Makefiles"  # Use Make generator

# Build
cmake --build build                 # Build the project
cmake --build build -j 8            # Parallel build (8 jobs)
cmake --build build --target app    # Build specific target
cmake --build build --clean-first   # Clean before building

# Install
cmake --install build               # Install to default prefix
cmake --install build --prefix /opt # Install to custom prefix
```

Build Types and Options

The `-D` flag sets CMake cache variables. These persist in `CMakeCache.txt` and control build configuration.

```
# Build types
cmake -B build -DCMAKE_BUILD_TYPE=Debug
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo
cmake -B build -DCMAKE_BUILD_TYPE=MinSizeRel

# Compiler selection
cmake -B build -DCMAKE_C_COMPILER=gcc -DCMAKE_CXX_COMPILER=g++
cmake -B build -DCMAKE_C_COMPILER=clang -DCMAKE_CXX_COMPILER=clang++

# Install prefix
cmake -B build -DCMAKE_INSTALL_PREFIX=/usr/local
cmake -B build -DCMAKE_INSTALL_PREFIX=$HOME/.local

# Project options (defined by option() in CMakeLists.txt)
cmake -B build -DBUILD_TESTS=ON
cmake -B build -DBUILD_SHARED_LIBS=ON
cmake -B build -DENABLE_FEATURE_X=OFF

# Multiple options
cmake -B build \
    -DCMAKE_BUILD_TYPE=Release \
    -DCMAKE_CXX_STANDARD=17 \
    -DBUILD_SHARED_LIBS=ON \
    -DBUILD_TESTS=OFF
```

Inspect and Debug

```
# List available targets
cmake --build build --target help

# List cache variables
cmake -B build -L                   # List non-advanced variables
cmake -B build -LA                  # List all variables
cmake -B build -LAH                 # List all with help strings

# Print variable value
cmake -B build -N -L | grep CMAKE_CXX

# Verbose build output
cmake --build build --verbose
cmake --build build -- VERBOSE=1    # Make-specific
```

Testing and Packaging

```
# Run tests
ctest --test-dir build              # Run all tests
ctest --test-dir build -V           # Verbose output
ctest --test-dir build -j 8         # Parallel testing
ctest --test-dir build -R "regex"   # Run matching tests
ctest --test-dir build --rerun-failed

# Create package
cmake --build build --target package
cpack --config build/CPackConfig.cmake
```

Project Setup

Every CMake project starts with basic configuration. Modern CMake recommends setting the C++ standard on targets rather than globally, which provides better control and avoids polluting the global namespace.

Minimal Project

The simplest CMake project requires only a few lines. The `cmake_minimum_required` ensures compatibility, while `project` defines metadata used throughout the build.

```
cmake_minimum_required(VERSION 3.16)
project(MyProject VERSION 1.0.0 LANGUAGES CXX)

add_executable(app main.cpp)
target_compile_features(app PRIVATE cxx_std_17)
```

Modern C++ Standard Setting

Using `target_compile_features` is preferred over global `CMAKE_CXX_STANDARD` because it applies only to specific targets and can require specific language features.

```
# Per-target (preferred)
target_compile_features(app PRIVATE cxx_std_17)
target_compile_features(app PRIVATE cxx_std_20)

# Or require specific features
target_compile_features(app PRIVATE cxx_constexpr cxx_lambdas)

# Global (legacy, avoid)
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
```

Build Commands

CMake separates configuration from building. The `-B` flag specifies the build directory, keeping source and build files separate (out-of-source build).

```
# Configure and build
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)

# Common build types
cmake -B build -DCMAKE_BUILD_TYPE=Debug
cmake -B build -DCMAKE_BUILD_TYPE=Release
cmake -B build -DCMAKE_BUILD_TYPE=RelWithDebInfo

# Specify generator
cmake -B build -G Ninja
cmake -B build -G "Unix Makefiles"

# Install
cmake --install build --prefix /usr/local
```

Targets and Properties

Modern CMake is target-centric. Use `target_*` commands instead of global variables like `CMAKE_CXX_FLAGS`. This approach provides better encapsulation and makes dependencies explicit.

Executables and Libraries

CMake supports multiple library types. Use `STATIC` for archives, `SHARED` for dynamic libraries, and `INTERFACE` for header-only libraries.

```
# Executable
add_executable(app main.cpp utils.cpp)

# Static library
add_library(mylib STATIC lib.cpp)

# Shared library
add_library(mylib SHARED lib.cpp)

# Header-only library
add_library(mylib INTERFACE)

# Let CMake decide (BUILD_SHARED_LIBS)
add_library(mylib lib.cpp)
```

Include Directories

The `target_include_directories` command specifies where the compiler should look for header files. Generator expressions like `$<BUILD_INTERFACE>` and `$<INSTALL_INTERFACE>` allow different paths during build and after installation. This is essential for creating relocatable packages that work both in the build tree and when installed to a system location.

```
target_include_directories(mylib
    PUBLIC
        $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
        $<INSTALL_INTERFACE:include>
    PRIVATE
        ${CMAKE_CURRENT_SOURCE_DIR}/src
)
```

Compile Options and Definitions

Compile flags and preprocessor definitions should be set per-target. Generator expressions enable conditional flags based on build type or compiler.

```
# Compile flags
target_compile_options(app PRIVATE
    -Wall -Wextra -Wpedantic
    $<$<CONFIG:Debug>:-g -O0>
    $<$<CONFIG:Release>:-O3>
)

# Preprocessor definitions
target_compile_definitions(app PRIVATE
    APP_VERSION="${PROJECT_VERSION}"
    $<$<CONFIG:Debug>:DEBUG_MODE>
)
```

Linking

The `target_link_libraries` command specifies libraries to link against and automatically handles transitive dependencies. When you link to a library, CMake propagates that library's PUBLIC include directories and compile definitions to your target. Libraries with `::` in their name (like `Threads::Threads`) are imported targets that carry all necessary usage requirements.

```
target_link_libraries(app
    PRIVATE
        mylib
        Threads::Threads
    PUBLIC
        Boost::boost
)
```

PUBLIC vs PRIVATE vs INTERFACE

These keywords control how properties propagate to dependent targets:

- `PRIVATE`: The property applies only to the target itself, not to anything that links to it. Use for implementation details like source files or internal dependencies.
- `PUBLIC`: The property applies to both the target and anything that links to it. Use for headers that consumers need to include.
- `INTERFACE`: The property applies only to targets that link to this one, not to the target itself. Use for header-only libraries.

Understanding these keywords is crucial for creating well-encapsulated libraries that don't leak unnecessary dependencies.

```
# PRIVATE: only for this target
# PUBLIC: for this target and dependents
# INTERFACE: only for dependents (header-only libs)

add_library(mylib STATIC lib.cpp)
target_include_directories(mylib PUBLIC include/)   # Consumers need headers
target_link_libraries(mylib PRIVATE internal_dep)   # Implementation detail
target_compile_definitions(mylib INTERFACE USE_MYLIB)  # Define for consumers
```

Dependency Management

Modern CMake provides multiple ways to manage external dependencies. FetchContent is preferred for most cases as it integrates seamlessly with the build.

FetchContent (Recommended)

FetchContent downloads and configures dependencies at configure time, making them available as regular CMake targets. Unlike ExternalProject (which builds at build time), FetchContent integrates dependencies into your build system so they can be used immediately with `target_link_libraries`. This is the recommended approach for most projects as it provides the simplest integration.

```
include(FetchContent)

FetchContent_Declare(
    fmt
    GIT_REPOSITORY https://github.com/fmtlib/fmt.git
    GIT_TAG 10.1.1
)
FetchContent_MakeAvailable(fmt)

target_link_libraries(app PRIVATE fmt::fmt)
```

Multiple Dependencies

Multiple dependencies can be declared together and made available with a single `FetchContent_MakeAvailable` call for efficiency.

```
include(FetchContent)

FetchContent_Declare(
    googletest
    GIT_REPOSITORY https://github.com/google/googletest.git
    GIT_TAG v1.14.0
)
FetchContent_Declare(
    nlohmann_json
    GIT_REPOSITORY https://github.com/nlohmann/json.git
    GIT_TAG v3.11.3
)

FetchContent_MakeAvailable(googletest nlohmann_json)

target_link_libraries(app PRIVATE nlohmann_json::nlohmann_json)
target_link_libraries(tests PRIVATE GTest::gtest_main)
```

find_package

Use `find_package` for system-installed libraries or when you want to use pre-built binaries instead of building from source. CMake supports two modes:

- **Config mode** (preferred): The library provides its own CMake config files (e.g., `fmtConfig.cmake`). These are typically installed alongside the library.

- **Module mode**: CMake provides a `Find*.cmake` module that knows how to locate the library. Used for libraries that don't provide CMake support.

Config mode is preferred because the library author knows best how to expose their targets.

```
# Config mode (preferred, library provides CMake config)
find_package(fmt REQUIRED CONFIG)
target_link_libraries(app PRIVATE fmt::fmt)

# Module mode (CMake provides Find*.cmake)
find_package(Threads REQUIRED)
find_package(OpenSSL REQUIRED)
target_link_libraries(app PRIVATE
    Threads::Threads
    OpenSSL::SSL
    OpenSSL::Crypto
)

# Optional dependency
find_package(Boost COMPONENTS filesystem)
if(Boost_FOUND)
    target_link_libraries(app PRIVATE Boost::filesystem)
endif()
```

Common find_package Examples

These examples show how to link against commonly used libraries that provide CMake config files or have built-in Find modules.

```
# Boost (header-only and compiled components)
find_package(Boost 1.70 REQUIRED COMPONENTS system filesystem)
target_link_libraries(app PRIVATE Boost::system Boost::filesystem)

# Boost header-only
find_package(Boost REQUIRED)
target_link_libraries(app PRIVATE Boost::boost)

# Protobuf
find_package(Protobuf REQUIRED)
target_link_libraries(app PRIVATE protobuf::libprotobuf)

# CURL
find_package(CURL REQUIRED)
target_link_libraries(app PRIVATE CURL::libcurl)

# OpenSSL
find_package(OpenSSL REQUIRED)
target_link_libraries(app PRIVATE OpenSSL::SSL OpenSSL::Crypto)

# ZLIB
find_package(ZLIB REQUIRED)
target_link_libraries(app PRIVATE ZLIB::ZLIB)
```

pkg-config Fallback

For libraries that don't provide CMake config files, pkg-config can be used as a fallback. The `IMPORTED_TARGET` option creates a proper CMake target.

```
find_package(PkgConfig REQUIRED)
pkg_check_modules(LIBSSH2 REQUIRED IMPORTED_TARGET libssh2)
target_link_libraries(app PRIVATE PkgConfig::LIBSSH2)
```

Project Structure

Organizing larger projects with subdirectories keeps code modular and maintainable. Each subdirectory can have its own CMakeLists.txt defining local targets.

Subdirectories

A typical project structure separates source, library, and test code into distinct directories, each with its own CMakeLists.txt.

```
project/
├── CMakeLists.txt
├── src/
│   ├── CMakeLists.txt
│   └── main.cpp
├── lib/
│   ├── CMakeLists.txt
│   ├── mylib.cpp
│   └── mylib.h
└── tests/
    ├── CMakeLists.txt
    └── test_mylib.cpp
```

Root CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.16)
project(MyProject VERSION 1.0.0 LANGUAGES CXX)

option(BUILD_TESTS "Build tests" ON)

add_subdirectory(lib)
add_subdirectory(src)

if(BUILD_TESTS)
    enable_testing()
    add_subdirectory(tests)
endif()
```

lib/CMakeLists.txt:

```
add_library(mylib mylib.cpp)
add_library(MyProject::mylib ALIAS mylib)

target_include_directories(mylib
    PUBLIC
        $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}>
        $<INSTALL_INTERFACE:include>
)
target_compile_features(mylib PUBLIC cxx_std_17)
```

src/CMakeLists.txt:

```
add_executable(app main.cpp)
target_link_libraries(app PRIVATE MyProject::mylib)
```

Generator Expressions

Generator expressions are evaluated during build system generation, enabling conditional configuration based on build type, compiler, platform, and more. They use the syntax `$<...>`.

Common Patterns

Generator expressions enable conditional logic without if() statements, making CMakeLists.txt cleaner and more portable.

```
# Build type
$<$<CONFIG:Debug>:-g>
$<$<CONFIG:Release>:-O3>
$<IF:$<CONFIG:Debug>,-g,-O3>

# Compiler
$<$<CXX_COMPILER_ID:GNU>:-Wall>
$<$<CXX_COMPILER_ID:Clang>:-Weverything>
$<$<CXX_COMPILER_ID:MSVC>:/W4>

# Platform
$<$<PLATFORM_ID:Linux>:-pthread>
$<$<PLATFORM_ID:Windows>:-DWIN32>

# Build vs install paths
$<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>
$<INSTALL_INTERFACE:include>
```

Cross-Platform Compile Options

Generator expressions make it easy to set compiler-specific flags without platform-

specific if() blocks.

```
target_compile_options(app PRIVATE
    $<$<CXX_COMPILER_ID:MSVC>:/W4 /WX>
    $<$<NOT:$<CXX_COMPILER_ID:MSVC>>:-Wall -Wextra -Werror>
)
```

Installation and Packaging

Making your library installable allows other projects to use it via `find_package`. This requires exporting targets and creating config files.

Basic Installation

The `install` command copies built artifacts to the installation prefix (default `/usr/local` on Unix). Including `GNUInstallDirs` provides standard directory variables (`CMAKE_INSTALL_BINDIR`, `CMAKE_INSTALL_LIBDIR`, etc.) that follow platform conventions, ensuring your project installs correctly on different systems.

```
include(GNUInstallDirs)

install(TARGETS mylib app
    EXPORT MyProjectTargets
    LIBRARY DESTINATION ${CMAKE_INSTALL_LIBDIR}
    ARCHIVE DESTINATION ${CMAKE_INSTALL_LIBDIR}
    RUNTIME DESTINATION ${CMAKE_INSTALL_BINDIR}
    INCLUDES DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
)

install(DIRECTORY include/
    DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}
)
```

Export for find_package

To allow other CMake projects to use your library with `find_package(MyProject)`, you need to export your targets and create config files. This involves:

1. Exporting targets to a `*Targets.cmake` file
2. Creating a `*Config.cmake` that includes the targets file
3. Optionally creating a `*ConfigVersion.cmake` for version checking

The namespace (e.g., `MyProject::`) prevents target name collisions and signals that the target is imported.

```
install(EXPORT MyProjectTargets
    FILE MyProjectTargets.cmake
    NAMESPACE MyProject::
    DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject
)

include(CMakePackageConfigHelpers)

configure_package_config_file(
    cmake/MyProjectConfig.cmake.in
    ${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfig.cmake
    INSTALL_DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject
)

write_basic_package_version_file(
    ${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfigVersion.cmake
    VERSION ${PROJECT_VERSION}
    COMPATIBILITY SameMajorVersion
)

install(FILES
    ${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfig.cmake
    ${CMAKE_CURRENT_BINARY_DIR}/MyProjectConfigVersion.cmake
    DESTINATION ${CMAKE_INSTALL_LIBDIR}/cmake/MyProject
)
```

cmake/MyProjectConfig.cmake.in:

```
@PACKAGE_INIT@
include(CMakeFindDependencyMacro)
# find_dependency(Boost REQUIRED)  # if needed
include("${CMAKE_CURRENT_LIST_DIR}/MyProjectTargets.cmake")
```

CPack for Distribution

CPack generates distributable packages (ZIP, TGZ, DEB, RPM) from your CMake project.

```
set(CPACK_PACKAGE_NAME ${PROJECT_NAME})
set(CPACK_PACKAGE_VERSION ${PROJECT_VERSION})
set(CPACK_PACKAGE_VENDOR "Your Name")
set(CPACK_GENERATOR "TGZ;ZIP")

# For DEB packages
set(CPACK_DEBIAN_PACKAGE_MAINTAINER "your@email.com")

include(CPack)
```

```
cmake --build build --target package
```

Testing

CTest is CMake's built-in testing framework that integrates with CMake to provide test discovery and execution. For GoogleTest users, the `gtest_discover_tests` function automatically registers all TEST() macros as individual CTest tests, enabling parallel execution and better reporting.

```
enable_testing()

add_executable(tests test_main.cpp)
target_link_libraries(tests PRIVATE mylib GTest::gtest_main)

include(GoogleTest)
gtest_discover_tests(tests)

# Or manual test registration
add_test(NAME MyTest COMMAND tests)
```

```
ctest --test-dir build -V
ctest --test-dir build -j$(nproc)
```

Useful Variables

CMake provides many built-in variables for accessing project information, paths, and build configuration. Understanding these variables helps write portable CMakeLists.txt files that work across different environments.

```
# Project info
${PROJECT_NAME}
${PROJECT_VERSION}
${PROJECT_SOURCE_DIR}
${PROJECT_BINARY_DIR}

# Current directory
${CMAKE_CURRENT_SOURCE_DIR}
${CMAKE_CURRENT_BINARY_DIR}
${CMAKE_CURRENT_LIST_DIR}

# Install paths (use with GNUInstallDirs)
${CMAKE_INSTALL_PREFIX}
${CMAKE_INSTALL_BINDIR}
${CMAKE_INSTALL_LIBDIR}
${CMAKE_INSTALL_INCLUDEDIR}

# Build info
${CMAKE_BUILD_TYPE}
${CMAKE_CXX_COMPILER_ID}
${CMAKE_SYSTEM_NAME}
```

Options and Cache Variables

Options allow users to customize the build without modifying CMakeLists.txt. Cache variables persist across CMake invocations (stored in `CMakeCache.txt`) and can be set via command line with `-D`. This is how users enable/disable features, specify paths, or configure build options.

```
# Boolean option
option(BUILD_SHARED_LIBS "Build shared libraries" ON)
option(BUILD_TESTS "Build tests" OFF)

# Cache variable with type
set(MY_OPTION "default" CACHE STRING "Description")
set(MY_PATH "/usr/local" CACHE PATH "Install path")

# Use in CMake
if(BUILD_TESTS)
    add_subdirectory(tests)
endif()
```

```
cmake -B build -DBUILD_TESTS=ON -DMY_OPTION=custom
```

Presets (CMake 3.19+)

CMakePresets.json provides reproducible, shareable build configurations that can be version-controlled with your project. It eliminates the need to remember complex command-line options and ensures all developers use consistent settings. Presets can define configure options, build settings, and test configurations.

```
{
    "version": 6,
    "configurePresets": [
        {
            "name": "debug",
            "binaryDir": "${sourceDir}/build/debug",
            "cacheVariables": {
                "CMAKE_BUILD_TYPE": "Debug"
            }
        },
        {
            "name": "release",
            "binaryDir": "${sourceDir}/build/release",
            "cacheVariables": {
                "CMAKE_BUILD_TYPE": "Release"
            }
        }
    ],
    "buildPresets": [
        {
            "name": "debug",
            "configurePreset": "debug"
        },
        {
            "name": "release",
            "configurePreset": "release"
        }
    ]
}
```

```
cmake --preset debug
cmake --build --preset debug
```

CUDA Support

CMake has native CUDA support since version 3.8, treating CUDA as a first-class language alongside C and C++. This eliminates the need for the legacy `FindCUDA` module. The `CUDAToolkit` package (CMake 3.17+) provides imported targets for CUDA libraries like cuBLAS and cuFFT.

Basic CUDA Project

Enable CUDA by adding it to the `LANGUAGES` list. CMake automatically finds the CUDA compiler (nvcc) and configures `.cu` file compilation.

```
cmake_minimum_required(VERSION 3.18)
project(cuda_example LANGUAGES CXX CUDA)

find_package(CUDAToolkit REQUIRED)

add_executable(app main.cu)
target_compile_features(app PRIVATE cuda_std_17)
target_link_libraries(app PRIVATE CUDA::cudart)
```

Mixed C++/CUDA

For projects with both `.cpp` and `.cu` files, enable separable compilation to allow device code in different translation units to call each other.

```
add_library(kernels kernels.cu)
set_target_properties(kernels PROPERTIES
    CUDA_SEPARABLE_COMPILATION ON
    CUDA_ARCHITECTURES "70;80;86;90"  # Volta, Ampere, Hopper
)

add_executable(app main.cpp)
target_link_libraries(app PRIVATE kernels CUDA::cudart)
```

CUDA Architectures

`CUDA_ARCHITECTURES` specifies GPU compute capabilities to target. Use semicolon-separated values for multiple architectures. Common values:

- 70: Volta (V100)
- 75: Turing (RTX 20xx)
- 80: Ampere (A100)
- 86: Ampere (RTX 30xx)
- 89: Ada Lovelace (RTX 40xx)
- 90: Hopper (H100)

```
# Per-target (preferred)
set_target_properties(app PROPERTIES CUDA_ARCHITECTURES "80;86;90")  # Ampere

# Global default
set(CMAKE_CUDA_ARCHITECTURES "80;86;90")

# Detect host GPU at configure time (CMake 3.24+)
set(CMAKE_CUDA_ARCHITECTURES native)
```

CUDA Libraries

The `CUDAToolkit` package provides imported targets for NVIDIA libraries.

```
find_package(CUDAToolkit REQUIRED)

target_link_libraries(app PRIVATE
    CUDA::cudart       # Runtime API
    CUDA::cublas       # Dense linear algebra
    CUDA::cufft        # Fast Fourier transforms
    CUDA::curand       # Random number generation
    CUDA::cusolver     # Direct solvers
    CUDA::cusparse     # Sparse linear algebra
    CUDA::nvml         # GPU monitoring and management
)
```